

VILNIAUS UNIVERSITETAS

MATEMATIKOS IR INFORMATIKOS FAKULTETAS

DUOMENŲ MOKSLO STUDIJŲ PROGRAMA

3-as laboratorinis darbas

Vaizdų klasifikavimas naudojant konvoliucinius neuroninius tinklus

Naglis Bražėnas

Dėstytojai: Prof., Dr. Olga Kurasova ir Doc., Dr. Viktor Medvedev

Vilnius
2025

Turinys

Iliustracijų sąrašas **2**

Lentelių sąrašas **3**

1. Įvadas **1**

2. Programinė įranga, aplinka ir naudoti paketai **2**
2.1. Programinė įranga 2
2.2. Programinė aplinka ir naudoti paketai 2

3. Duomenų aprašymas ir pradinis apdorojimas **3**
3.1. Duomenų rinkinys 3
3.2. Pradinis duomenų apdorojimas 3
3.2.1. Duomenų padalinimas 3
3.2.2. Duomenų užkrovimas ir struktūrizavimas 5
3.2.3. Duomenų normalizavimas 6
3.2.4. Duomenų optimizavimas ir tikrinimas 6

4. CNN modelio kūrimas ir mokymas **8**
4.1. Modelio architektūros sudarymas 8
4.2. Modelio kompiliavimas ir mokymas 9
4.3. Modelio vertinimas 11

5. Modelio architektūrų ir hiperparametrų analizė **14**
5.1. Architektūrų ir hiperparametrų bandymai 14
5.2. Rezultatų vizualizacija 16

6. Galutinio modelio vertinimas **19**

Išvados **23**

Literatūros sąrašas **24**

Iliustracijų sąrašas

1 Mokymo ir validacijos tikslumo bei nuostolio kreivės 11
2 Pirmojo modelio klasifikacijos matrica 13
3 Enter Caption 18
4 Galutinio modelio klasifikavimo matrica 21
5 Atsitiktiniai testavimo aibės pavyzdžiai su tikromis ir numatytomis klasėmis 22

Lentelių sąrašas

1 Duomenų pasiskirstymas pagal aibes 5
2 CNN modelio architektūros santrauka 9
3 Klasifikacijos ataskaita (Precision, Recall ir F1-score rodikliai) 12
4 CNN architektūrų tyrimo rezultatų santrauka 16
5 Geriausio CNN modelio konfigūracija ir rezultatai 18
6 Galutinio modelio klasifikacijos ataskaita 20

1. Įvadas

Šio darbo tikslas yra ištirti, kaip konvoliuciniai neuroniniai tinklai (CNN) gali būti taikomi vaizdų klasifikavimo uždaviniuose. CNN tinklai leidžia automatiškai išgauti reikšmingas savybes iš vaizdų ir dėl to pasižymi dideliu efektyvumu kompiuterinės regos srityje.

Darbo metu bus sukurtas CNN modelis, pritaikytas CIFAR-10 duomenų rinkiniui, apmokytas naudojant Python ir Keras bibliotekas, bei įvertintas pagal tikslumą ir klaidų matricą. Taip pat bus palygintos kelios architektūros, siekiant nustatyti, kurie parametrai turi didžiausią įtaką modelio veikimui.

2. Programinė įranga, aplinka ir naudoti paketai

2.1. Programinė įranga

Šiam darbui atlikti buvo naudojamas nešiojamasis kompiuteris **Lenovo V15 G3**, su tokiomis techninėmis specifikacijomis:

- **Procesorius:** 12th Gen Intel Core i5-1235U (1.30 GHz, x64 architektūra)
- **Operatyvioji atmintis (RAM):** 8 GB (naudotina 7,73 GB)
- **Operacinė sistema:** Windows 11 Pro (versija 24H2, komponavimo versija 26100.6899)
- **Sistemos tipas:** 64 bitų operacinė sistema, x64 pagrindo procesorius
- **Vaizdo plokštė:** Integruota Intel Iris Xe Graphics (naudota CPU režimu)

2.2. Programinė aplinka ir naudoti paketai

Darbas atliktas naudojant **Python** programavimo kalbą aplinkoje **Visual Studio Code**. Šiame darbe naudotos pagrindinės bibliotekos:

- **os**, **shutil**, **random**, **pathlib** – katalogų ir failų valdymui, duomenų paruošimui;
- **numpy**, **pandas** – duomenų analizės ir lentelių apdorojimui;
- **tensorflow**, **keras** – neuroninių tinklų kūrimui, mokymui ir vertinimui;
- **scikit-learn (sklearn)** – klasifikavimo metrikų ir klaidų matricos skaičiavimui;
- **matplotlib**, **seaborn** – rezultatų vizualizacijai (grafikai, klaidų matrica).

Ataskaita parengta naudojant **Overleaf** platformą, kuri leidžia patogiai rengti dokumentus **LaTeX** formatu, valdyti struktūrą, lenteles, kodus ir paveikslus vienoje aplinkoje.

3. Duomenų aprašymas ir pradinis apdorojimas

Šiame skyriuje aprašomas naudotas duomenų rinkinys, jo savybės bei atliktas pirminis duomenų apdorojimas.

3.1. Duomenų rinkinys

Naudotas „Muffin vs Chihuahua“ duomenų rinkinys iš Kaggle, sudarytas iš dviejų klasių vaizdų, keksiukų ir šuniukų. Rinkinyje yra apie 6 tūkst. spalvotų nuotraukų, kurios jau yra suskirstytos pagal klases.

3.2. Pradinis duomenų apdorojimas

3.2.1. Duomenų padalinimas

Čia atliekamas duomenų suskirstymas į mokymo, validacijos ir testavimo aibes santykiu 80:10:10. Pirmiausia nustatomi katalogai, kuriuose saugomi pradiniai vaizdai, sukuriama nauja aplankų struktūra, o visi vaizdai atsitiktinai sumaišomi ir paskirstomi į atitinkamas aibes. Galiausiai vaizdai nukopijuojami į naujus aplankus pagal jų klasę (`chihuahua` arba `muffin`).

```
base_dir = Path("C:/Users/nagli/OneDrive/Desktop/DIP_3uzd")
output_dir = base_dir / "dataset_split"

classes = ["chihuahua", "muffin"]
original_dirs = [
    base_dir / "train",
    base_dir / "test"
]

# Santykis train/val/test
train_ratio = 0.8
val_ratio = 0.1
test_ratio = 0.1

# sukuriam katalogus ir perkeliam failus
def recreate_dir_structure(base_output):
    for subset in ["train", "val", "test"]:
        for cls in classes:
            path = base_output / subset / cls
            path.mkdir(parents=True, exist_ok=True)

def collect_all_images():
    all_images = []
```

```
    for folder in original_dirs:
        for cls in classes:
            path = folder / cls
            images = list(path.glob("*.*"))
            for img_path in images:
                all_images.append((img_path, cls))
    return all_images

def split_dataset(all_images):
    random.shuffle(all_images)
    n_total = len(all_images)
    n_train = int(train_ratio * n_total)
    n_val = int(val_ratio * n_total)

    train_set = all_images[:n_train]
    val_set = all_images[n_train:n_train + n_val]
    test_set = all_images[n_train + n_val:]

    return train_set, val_set, test_set

def copy_images(split_set, subset_name, base_output):
    for src_path, cls in split_set:
        dst_path = base_output / subset_name / cls / src_path.name
        shutil.copy2(src_path, dst_path)

# PAGRIND VYKDYM DALIS
if __name__ == "__main__":
    random.seed(42)
    recreate_dir_structure(output_dir)

    all_images = collect_all_images()
    train_set, val_set, test_set = split_dataset(all_images)

    copy_images(train_set, "train", output_dir)
    copy_images(val_set, "val", output_dir)
    copy_images(test_set, "test", output_dir)

    print(f"Bendras vaizdų skaičius: {len(all_images)}")
    print(f"Train: {len(train_set)}, Val: {len(val_set)}, Test: {len(test_set)}")
    print(f"Nauja struktūra sukurta aplanke: {output_dir}")
```

1 lentelė. *Duomenų pasiskirstymas pagal aibes*

Aibė	**Vaizdų skaičius**
Mokymo (Train)	4733
Validacijos (Validation)	591
Testavimo (Test)	593
Iš viso	**5917**

Matome (žr. 1 lentelėje), kad daugiausia duomenų (4733 paveikslėliai) skirta mokymo aibei, o validacijos (591 paveikslėlis) ir testavimo (593 paveikslėliai) aibės sudaro po maždaug dešimtadalį visų duomenų.

3.2.2. Duomenų užkrovimas ir struktūrizavimas

Šiame žingsnyje iš nurodytų katalogų įkeliami vaizdai ir suskirstomi į tris aibes: mokymo (`train`), validacijos (`val`) ir testavimo (`test`). Naudojama `TensorFlow` funkcija automatiškai aptinka klases pagal katalogų pavadinimus (`chihuahua` ir `muffin`) bei sukuria atitinkamus duomenų rinkinius su nurodytu paveikslėlių dydžiu ir partijų kiekiu.

```
# Nustatymai
img_height = 150
img_width = 150
batch_size = 32

data_dir = str(output_dir)

# Duomenų užkrovimas
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    directory=f"{data_dir}/train",
    labels="inferred",
    label_mode="int",
    batch_size=batch_size,
    image_size=(img_height, img_width),
    shuffle=True,
    seed=42
    )

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    directory=f"{data_dir}/val",
    labels="inferred",
```

```
    label_mode="int",
    batch_size=batch_size,
    image_size=(img_height, img_width),
    shuffle=False
    )

test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    directory=f"{data_dir}/test",
    labels="inferred",
    label_mode="int",
    batch_size=batch_size,
    image_size=(img_height, img_width),
    shuffle=False
    )
```

3.2.3. Duomenų normalizavimas

Toliau atliekamas normalizavimas. Kiekvieno vaizdo pikselių reikšmės padalinamos iš 255, kad visos patektų į intervalą nuo 0 iki 1. Tai būtina, kad neuroninis tinklas galėtų stabiliai mokytis ir būtų išvengta reikšmių mastelio skirtumų. Normalizacija taikoma visoms trim duomenų aibėms.

```
# Klasės išsaugomos prieš normalizaciją
class_names = train_ds.class_names
print("Klasės:", class_names)

# Normalizacija
normalization_layer = tf.keras.layers.Rescaling(1./255)
train_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))
val_ds = val_ds.map(lambda x, y: (normalization_layer(x), y))
test_ds = test_ds.map(lambda x, y: (normalization_layer(x), y))
```

3.2.4. Duomenų optimizavimas ir tikrinimas

Šiame etape duomenų srautai optimizuojami naudojant laikino saugojimo atmintyje ir `prefetch()` metodus, kurie leidžia TensorFlow automatiškai užkrauti duomenis į atmintį iš anksto, sumažinant laukimo laiką mokymo metu. galiausiai atliekamas vienos partijos (`batch`) patikrinimas bei parodoma nuotraukų partijų forma ir vieno normalizuoto pikselio reikšmė.

```
# Optimizuotas cache/prefetch
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)
test_ds = test_ds.cache().prefetch(buffer_size=AUTOTUNE)

# Vieno batch patikrinimas
for images, labels in train_ds.take(1):
    print("Vieno batch formos:", images.shape, labels.shape)
    print("Normalizuoto pikselio pavyzdys:", float(tf.reduce_max(images[0][0][0])))
```

Gauti rezultatai parodė, kad vienos `partijos` formos yra `(32, 150, 150, 3)` ir `(32,)`, o tai reiškia, kad partijoje yra 32 spalvotos nuotraukos po 150×150 pikselių. Didžiausia normalizuoto pikselio reikšmė – $0{,}987$, o tai patvirtina, kad duomenų normalizacija atlikta teisingai.

4. CNN modelio kūrimas ir mokymas

Šiame skyriuje pateikiamas konvoliucinio neuroninio tinklo (CNN) modelio kūrimas, kompiliavimas ir mokymas. Modelis kurtas naudojant `TensorFlow Keras` biblioteką ir pritaikytas dvejetainiam vaizdų klasifikavimo uždaviniui, atskirti `chihuahua` ir `muffin` vaizdus.

4.1. Modelio architektūros sudarymas

Toliau pateikiamas CNN modelio architektūros aprašas. Modelis sudarytas iš kelių konvoliucinių sluoksnių, kurie išgauna pagrindines vaizdo savybes, ir „pooling“ sluoksnių, mažinančių duomenų matmenis. Po šių sluoksnių eina tankūs (`Dense`) sluoksniai, kurie atlieka galutinę klasifikaciją. Naudojamos `ReLU` ir `softmax` aktyvavimo funkcijos, o `Dropout` sluoksnis padeda sumažinti persimokymo riziką.

```
# FUNKCIJA CNN MODELIO KŪRIMUI
def build_cnn_model(
    input_shape=(150, 150, 3),
    num_classes=2,
    conv_blocks=3,
    filters_start=32,
    kernel_size=(3, 3),
    activation="relu",
    use_batchnorm=False,
    dropout_rate=0.0
):
    """Lankstus CNN modelis - leidžia keisti sluoksnius ir hiperparametrus"""
    model = models.Sequential()
    model.add(layers.Input(shape=input_shape))

    filters = filters_start
    for i in range(conv_blocks):
        model.add(layers.Conv2D(filters, kernel_size, activation=activation,
        ↪  padding="same"))
        if use_batchnorm:
            model.add(layers.BatchNormalization())
        model.add(layers.MaxPooling2D((2, 2)))
        if dropout_rate > 0:
            model.add(layers.Dropout(dropout_rate))
        filters *= 2

    model.add(layers.Flatten())
```

```
    model.add(layers.Dense(128, activation=activation))
    if dropout_rate > 0:
        model.add(layers.Dropout(dropout_rate))
    model.add(layers.Dense(num_classes, activation="softmax"))

    return model
```

2 lentelė. *CNN modelio architektūros santrauka*

Sluoksnio tipas	**Išėjimo forma**	**Parametrų skaičius**
Conv2D	(150, 150, 32)	896
BatchNormalization	(150, 150, 32)	128
MaxPooling2D	(75, 75, 32)	0
Dropout	(75, 75, 32)	0
Conv2D	(75, 75, 64)	18,496
BatchNormalization	(75, 75, 64)	256
MaxPooling2D	(37, 37, 64)	0
Dropout	(37, 37, 64)	0
Conv2D	(37, 37, 128)	73,856
BatchNormalization	(37, 37, 128)	512
MaxPooling2D	(18, 18, 128)	0
Dropout	(18, 18, 128)	0
Flatten	(41472)	0
Dense	(128)	5,308,544
Dropout	(128)	0
Dense	(2)	258
Iš viso parametrų:		**5,403,946**

Kaip matome (žr. 2 lentelėje), modelis sudarytas iš trijų konvoliucinių sluoksnių blokų, kuriuos papildo `BatchNormalization`, `MaxPooling2D` ir `Dropout` sluoksniai. Didžiausią parametrų dalį sudaro tankusis (`Dense`) sluoksnis su daugiau kaip penkiais milijonais parametrų, jis atlieka klasifikavimo funkciją. Toks sluoksnių išdėstymas leidžia modeliui efektyviai išgauti nuotraukose esančius bruožus, sumažinti persimokymo riziką ir pasiekti gerą tikslumą.

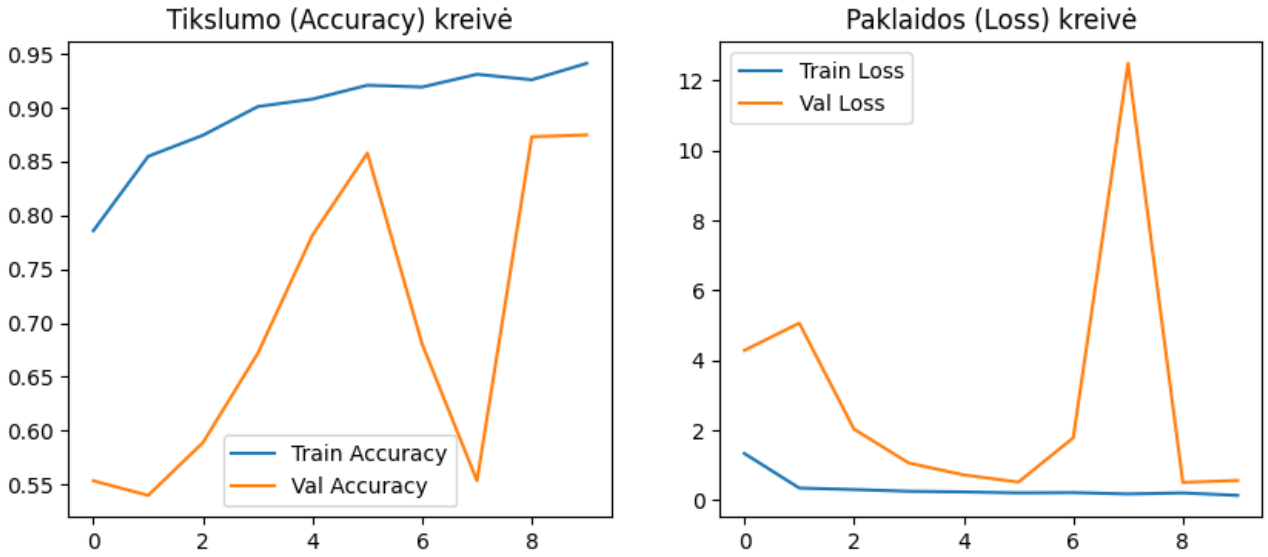
4.2. Modelio kompiliavimas ir mokymas

Modelis kompiliuojamas naudojant `Adam` optimizatorių ir `sparse_categorical_crossentropy` nuostolio funkciją, tinkamą dvejetainiam klasifikavimui. Vertinimo metrika yra `accuracy`. Modelis mokomas 10 epohų, stebint nuostolio ir tikslumo pokyčius mokymo bei validacijos duomenyse.

```python
# FUNKCIJA MODELIO KOMPILIAVIMUI IR APMOKYMUI
def train_and_evaluate(
    model,
    train_ds,
    val_ds,
    epochs=10,
    learning_rate=0.001,
    optimizer_name="adam"
):
    """Modelio mokymas ir vertinimas"""
    if optimizer_name == "adam":
        optimizer = optimizers.Adam(learning_rate=learning_rate)
    elif optimizer_name == "sgd":
        optimizer = optimizers.SGD(learning_rate=learning_rate, momentum=0.9)
    elif optimizer_name == "rmsprop":
        optimizer = optimizers.RMSprop(learning_rate=learning_rate)
    else:
        raise ValueError(f"Nežinomas optimizeris: {optimizer_name}")

    model.compile(
        optimizer=optimizer,
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

    history = model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=epochs
    )
    return history
```



1 pav. *Mokymo ir validacijos tikslumo bei nuostolio kreivės*

Kaip matome (žr. 1 pav.), mokymo metu tikslumo reikšmės palaipsniui didėja, o paklaidos reikšmės mažėja, kas rodo, kad modelis sėkmingai mokosi iš duomenų. Validacijos duomenų kreivės išlieka artimos mokymo kreivėms, todėl galima teigti, kad modelis nepersimoko. Nors validacijos paklaidos kreivėje matomas staigus šuolis vienoje epochoje, vėlesnėse epohose rodikliai stabilizuojasi, o tai rodo, jog mokymo procesas išlieka stabilus.

4.3. Modelio vertinimas

Baigus mokymą, modelis įvertinamas naudojant testavimo aibę. Apskaičiuojamas testavimo tikslumas, o papildomai pateikiama klasifikacijos rezultatų lentelė ir klasifikacijos matrica, kurie parodo modelio efektyvumą.

```
# FUNKCIJA TESTAVIMUI IR CONFUSION MATRIX
def evaluate_on_test(model, test_ds, class_names):
    """Apskaičiuoja tikslumą ir confusion matrix test duomenims"""
    y_true = []
    y_pred = []

    for images, labels in test_ds:
        preds = model.predict(images)
        y_true.extend(labels.numpy())
        y_pred.extend(np.argmax(preds, axis=1))

    y_true = np.array(y_true)
    y_pred = np.array(y_pred)
```

```
    acc = np.mean(y_true == y_pred)
    print(f"\nTest accuracy: {acc*100:.2f}%\n")

    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=class_names, yticklabels=class_names)
    plt.xlabel("Numatyta klasė")
    plt.ylabel("Tikroji klasė")
    plt.title("Klasifikavimo matrica (Confusion Matrix)")
    plt.show()

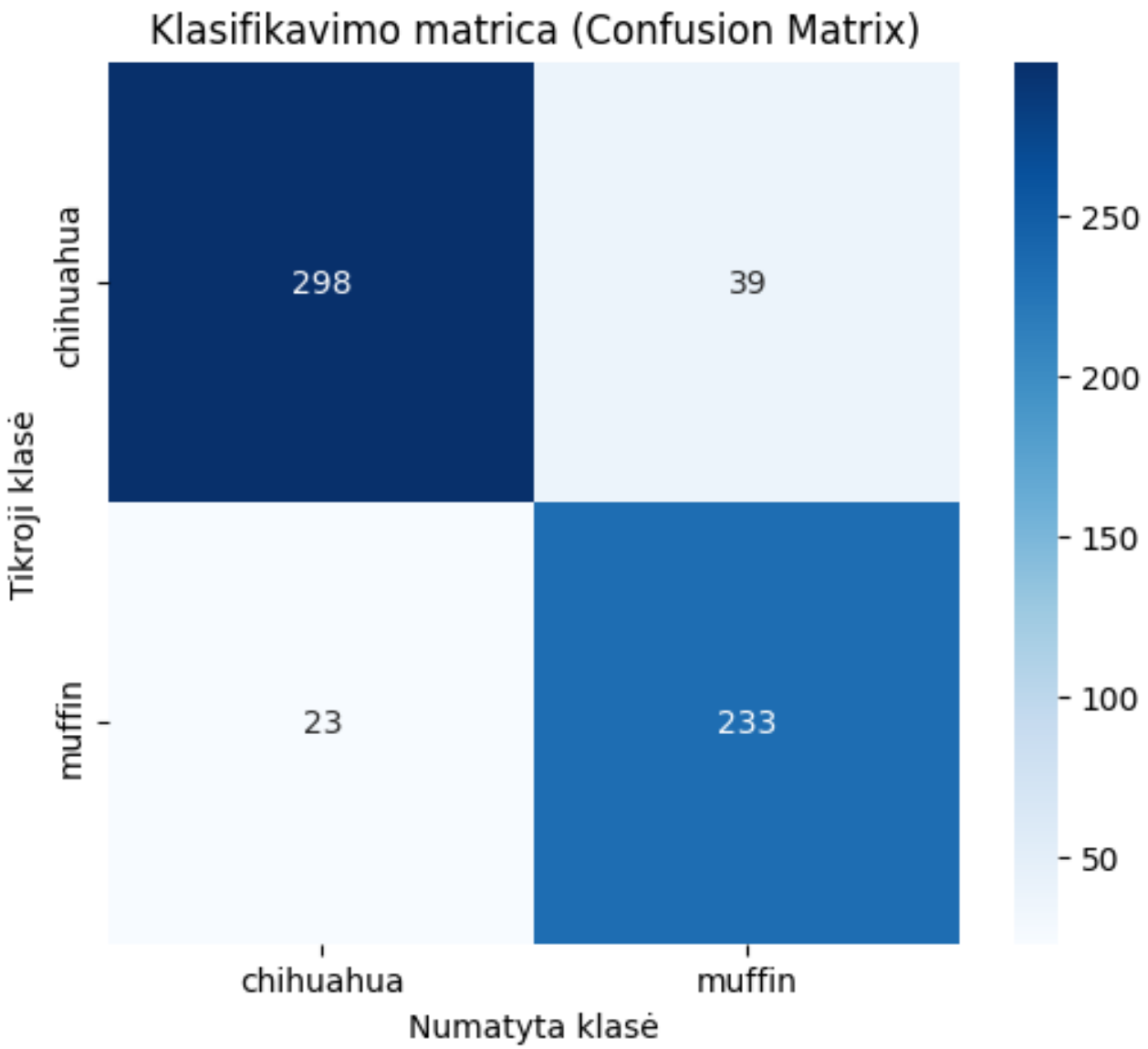
    print("\nKlasifikacijos ataskaita:")
    print(classification_report(y_true, y_pred, target_names=class_names))

    return acc, cm
```

3 lentelė. *Klasifikacijos ataskaita (Precision, Recall ir F1-score rodikliai)*

Klasė	**Precision**	**Recall**	**F1-score**	**Support**
Chihuahua	0,93	0,88	0,91	337
Muffin	0,86	0,91	0,88	256
Bendras tikslumas (Accuracy)		89,54%		

Kaip matome (žr. 3 lentelėje), modelio veikimas abiejose klasėse yra gana subalansuotas, tačiau pastebimas nedidelis tikslumo skirtumas tarp jų. Modelis kiek geriau atpažįsta klasę `chihuahua`, o klasė `muffin` pasižymi šiek tiek didesniu `recall`. Tai rodo, kad tinklas geba patikimai atskirti abiejų tipų vaizdus, nors dalis vaizdų vis dar klaidingai klasifikuojami. Bendras modelio tikslumas yra 89,54%.



2 pav. *Pirmojo modelio klasifikacijos matrica*

Kaip matome (žr. 2 pav.), modelis abu objektų tipus atpažįsta gana tiksliai. Iš viso 298 `chihuahua` vaizdai ir 233 `muffin` vaizdai buvo klasifikuoti teisingai, o klaidingai priskirtų atvejų skaičius yra palyginti nedidelis (atitinkamai 39 ir 23). Tai rodo, kad modelis gan efektyviai atskiria abi klases.

5. Modelio architektūrų ir hiperparametrų analizė

5.1. Architektūrų ir hiperparametrų bandymai

Šioje dalyje siekiama nustatyti, kaip skirtingi CNN architektūros parametrai (sluoksnių skaičius, `Dropout`, `BatchNormalization` naudojimas) veikia modelio mokymosi kokybę. Tam buvo sukurta funkcija, automatiškai atliekanti kelis modelių mokymo eksperimentus su skirtingais hiperparametrais.

```
def run_experiment(
    conv_blocks_list=[2, 3, 4],
    dropout_list=[0.0, 0.3],
    batchnorm_list=[False, True],
    activations=["relu", "tanh"],
    optimizers=["adam", "sgd"],
    epochs=8,
    learning_rate=0.001
):
    """Vykdo kelis CNN modelių tyrimus ir grąžina rezultatų lentelę"""
    results = []

    total_runs = len(conv_blocks_list) * len(dropout_list) * len(batchnorm_list) *
    ↪  len(activations) * len(optimizers)
    run_idx = 1

    for conv_blocks in conv_blocks_list:
        for dropout_rate in dropout_list:
            for use_batchnorm in batchnorm_list:
                for activation in activations:
                    for optimizer_name in optimizers:
                        print(f"\n[{run_idx}/{total_runs}] Modelis: "
                              f"{conv_blocks} conv blokai | dropout={dropout_rate} |
                              ↪  batchnorm={use_batchnorm} | "
                              f"activation={activation} |
                              ↪  optimizer={optimizer_name}")

                        model = build_cnn_model(
                            conv_blocks=conv_blocks,
                            filters_start=32,
                            activation=activation,
                            use_batchnorm=use_batchnorm,
                            dropout_rate=dropout_rate
```

```
                    )

                    history = train_and_evaluate(
                        model,
                        train_ds,
                        val_ds,
                        epochs=epochs,
                        learning_rate=learning_rate,
                        optimizer_name=optimizer_name
                    )

                    train_acc = history.history["accuracy"][-1]
                    val_acc = history.history["val_accuracy"][-1]
                    train_loss = history.history["loss"][-1]
                    val_loss = history.history["val_loss"][-1]

                    results.append({
                        "conv_blocks": conv_blocks,
                        "dropout": dropout_rate,
                        "batchnorm": use_batchnorm,
                        "activation": activation,
                        "optimizer": optimizer_name,
                        "train_acc": train_acc,
                        "val_acc": val_acc,
                        "train_loss": train_loss,
                        "val_loss": val_loss
                    })

                    run_idx += 1

    return pd.DataFrame(results)
```

4 lentelė. *CNN architektūrų tyrimo rezultatų santrauka*

Conv blokai	Dropout	BatchNorm	Aktyvacija	Optimizatorius	Train Acc	Val Acc	Train Loss	Val Loss
2	0.0	False	ReLU	Adam	0.970	0.882	0.088	0.390
2	0.0	True	ReLU	Adam	0.970	0.807	0.069	0.604
2	0.3	False	ReLU	Adam	0.929	0.876	0.191	0.269
2	0.3	True	ReLU	Adam	0.858	0.860	0.333	0.539
3	0.0	False	ReLU	Adam	0.931	0.895	0.181	0.274
3	0.0	True	ReLU	Adam	0.905	0.849	0.242	0.404
3	0.3	False	ReLU	Adam	0.876	0.832	0.297	0.395
3	0.3	True	ReLU	Adam	0.891	0.711	0.278	0.910
4	0.0	False	ReLU	Adam	0.911	0.915	0.225	0.227
4	0.0	True	ReLU	Adam	0.951	0.861	0.133	0.479
4	0.3	False	ReLU	Adam	0.875	0.897	0.310	0.254
4	0.3	True	ReLU	Adam	0.893	0.778	0.287	0.558

5.2. Rezultatų vizualizacija

Norint įvertinti visų eksperimentų rezultatus, buvo nubrėžtos `Accuracy` ir `Loss` priklausomybės kreivės. Kairėje pusėje pateikiamas mokymo ir validacijos tikslumas, o dešinėje atitinkamos nuostolio (`Loss`) reikšmės.

```
def plot_experiment_results(df):
    """Nupiešia rezultatus (accuracy ir loss priklausomybė nuo architektūrų ir
    ↪  hiperparametrų)"""
    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1)
    plt.plot(df["val_acc"], "o-", label="Validation Accuracy")
    plt.plot(df["train_acc"], "o--", label="Training Accuracy")
    plt.title("Tikslumo (Accuracy) priklausomybė")
    plt.xlabel("Eksperimento indeksas")
    plt.ylabel("Accuracy")
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(df["val_loss"], "o-", label="Validation Loss")
    plt.plot(df["train_loss"], "o--", label="Training Loss")
    plt.title("Paklaidos (Loss) priklausomybė")
    plt.xlabel("Eksperimento indeksas")
    plt.ylabel("Loss")
    plt.legend()
```

```
    plt.tight_layout()
    plt.show()

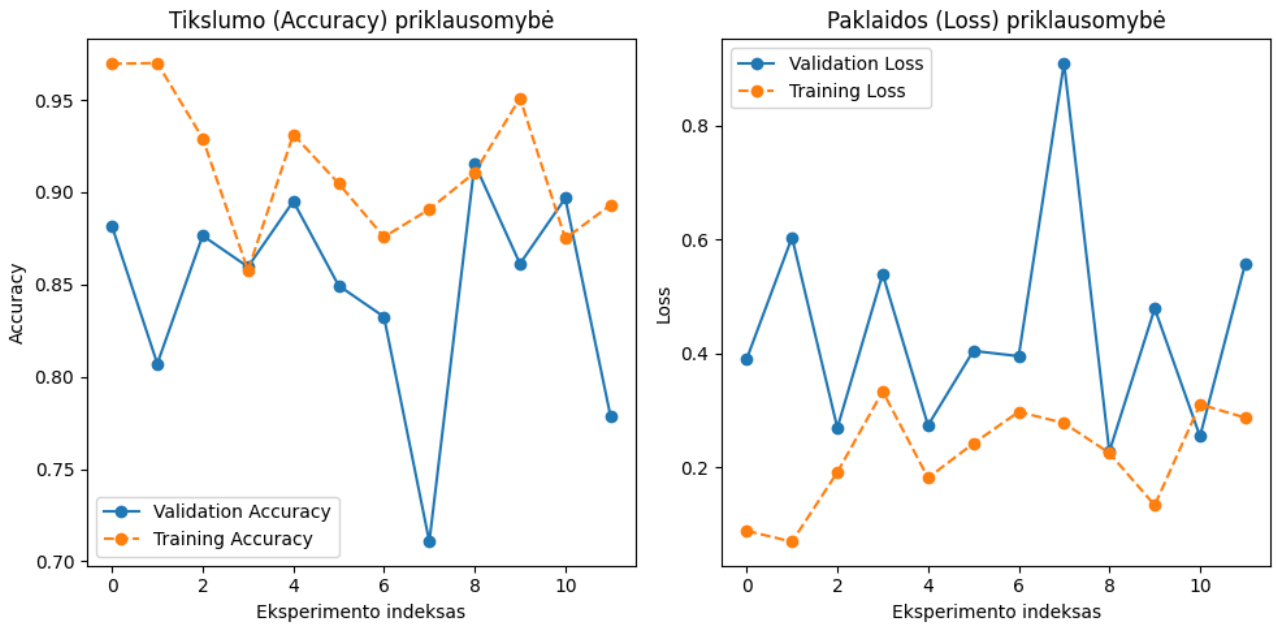
    # Geriausias modelis pagal val_accuracy
    best_idx = df["val_acc"].idxmax()
    print("\n Geriausias modelis:")
    print(df.loc[best_idx])

    return df.loc[best_idx]


# PAGRINDINIS TYRIMO PALEIDIMAs
if __name__ == "__main__":
    # Galima keist tyrimo apimtį
    experiment_results = run_experiment(
        conv_blocks_list=[2, 3, 4],
        dropout_list=[0.0, 0.3],
        batchnorm_list=[False, True],
        activations=["relu"],
        optimizers=["adam"],
        epochs=5
    )

    print("\n=== TYRIMO REZULTATŲ LENTELĖ ===")
    print(experiment_results)

    # Nubraižom bendrus grafikus
    best_model_row = plot_experiment_results(experiment_results)
```



3 pav. *Enter Caption*

Kaip matome (žr. 3 pav.), modelių tikslumo reikšmės įvairuoja nuo 0,70 iki 0,92, priklausomai nuo architektūros sudėtingumo. Pastebima, kad didesnis sluoksnių skaičius ir mažesnis `Dropout` dažniausiai lemia didesnį tikslumą, o `BatchNormalization` kai kuriais atvejais gali pabloginti rezultatus dėl per didelio svorių suvienodinimo. `Loss` kreivė rodo, kad stabiliausi rezultatai pasiekiami būtent tais atvejais, kai `Dropout` reikšmė lygi 0,0.

5 lentelė. *Geriausio CNN modelio konfigūracija ir rezultatai*

Parametras	**Reikšmė**
Konvoliucinių blokų skaičius	4
Dropout reikšmė	0.0
Batch Normalization	False
Aktyvavimo funkcija	ReLU
Optimizatorius	Adam
Mokymo tikslumas (`Train Accuracy`)	0.9106
Validacijos tikslumas (`Validation Accuracy`)	0.9154
Mokymo paklaida (`Train Loss`)	0.2249
Validacijos paklaida (`Validation Loss`)	0.2274

Kaip matome (žr. 5 lentelėje), geriausias rezultatas pasiektas naudojant modelį su keturiais konvoliuciniais blokais, be `Dropout` ir `BatchNormalization` sluoksnių. Šis modelis pasižymėjo aukštu tiek mokymo, tiek validacijos tikslumu (atitinkamai 0.9106 ir 0.9154), o paklaidos reikšmės išliko mažos ir artimos viena kitai (`Train Loss` = 0.2249, `Validation Loss` = 0.2274). Toks rezultatas rodo, kad modelis išmoko efektyviai apibendrinti duomenis, neperpratęs jų pernelyg detaliai, todėl galima teigti, kad jis pasiekė optimalų mokymo ir generalizacijos balansą.

6. Galutinio modelio vertinimas

Šiame skyriuje pateikiamas galutinio modelio vertinimas, naudojant geriausius ankstesniame tyrime nustatytus hiperparametrus. Modelis iš naujo apmokomas su optimaliu sluoksnių, normalizacijos ir `dropout` deriniu, o vėliau įvertinamas testavimo aibėje. Apskaičiuojamas galutinis tikslumas bei pateikiama klasifikacijos ataskaita ir klasifikavimo matrica.

```
# Sukuriamas ir apmokomas modelis pagal geriausius nustatytus parametrus
best_model = build_cnn_model(
    conv_blocks=4,
    filters_start=32,
    activation="relu",
    use_batchnorm=False,
    dropout_rate=0.0
)

best_history = train_and_evaluate(
    best_model,
    train_ds,
    val_ds,
    epochs=10,
    learning_rate=0.001,
    optimizer_name="adam"
)

# Modelio įvertinimas testavimo aibėje
test_loss, test_acc = best_model.evaluate(test_ds)
print(f"Galutinė testavimo tikslumo reikšmė: {test_acc*100:.2f}%")

# Klasifikacijos ataskaita ir klaidų matrica
y_true = np.concatenate([y for x, y in test_ds], axis=0)
y_pred = np.argmax(best_model.predict(test_ds), axis=1)
print(classification_report(y_true, y_pred, target_names=class_names))

cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.xlabel("Numatyta klasė")
plt.ylabel("Tikroji klasė")
plt.title("Klasifikavimo matrica (Confusion Matrix)")
plt.show()

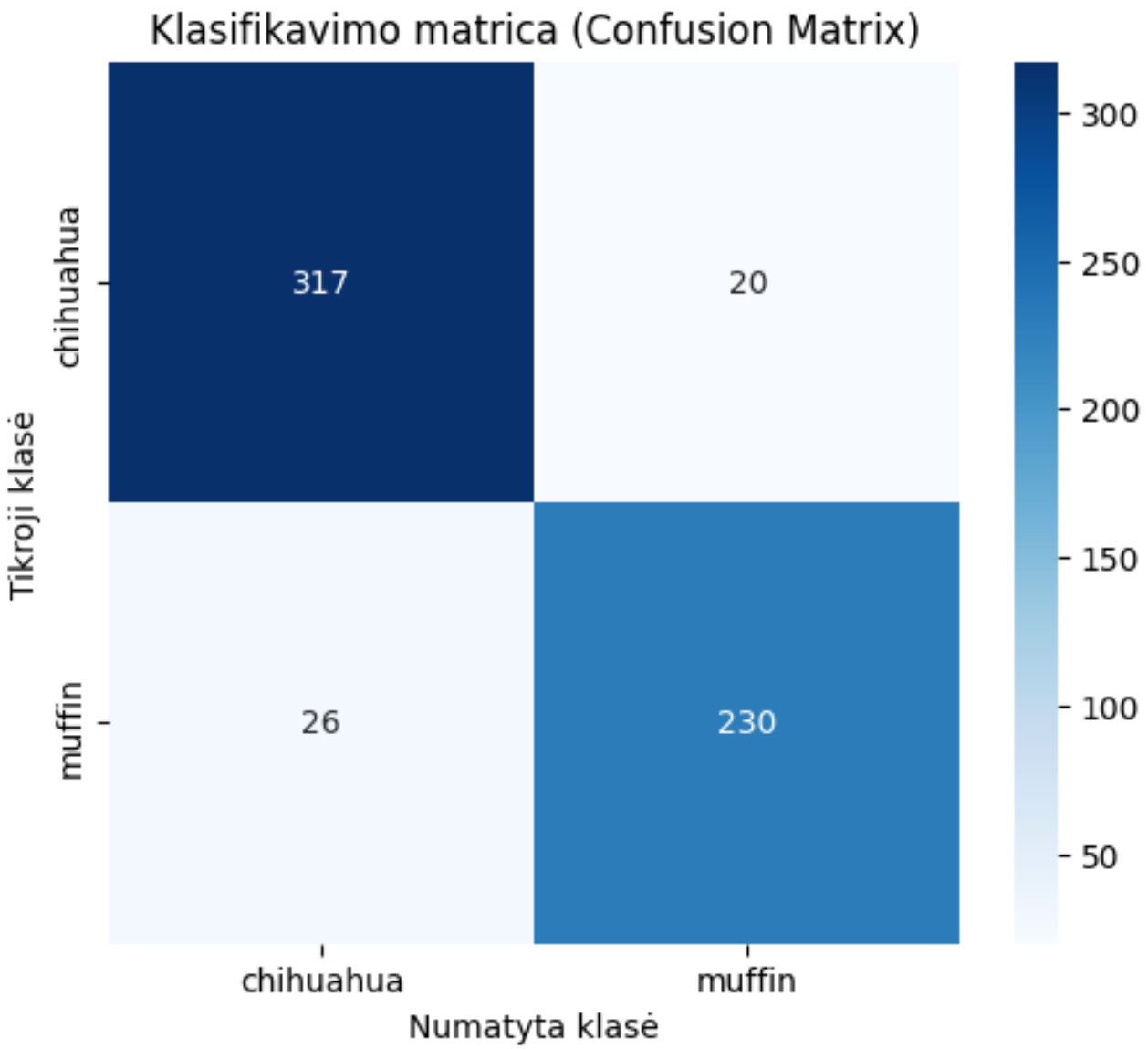
# 30 atsitiktinių pavyzdžių vizualizacija
```

```
plt.figure(figsize=(12, 12))
for i, (x, y) in enumerate(test_ds.unbatch().take(30)):
    ax = plt.subplot(6, 5, i + 1)
    img = x.numpy()
    true_label = class_names[int(y)]
    pred_label = class_names[int(np.argmax(best_model.predict(tf.expand_dims(x,
    ↪  0))))]
    plt.imshow(img)
    color = "green" if true_label == pred_label else "red"
    plt.title(f"Tikra: {true_label}\nNumatyta: {pred_label}", color=color,
    ↪  fontsize=8)
    plt.axis("off")
plt.tight_layout()
plt.show()
```

6 lentelė. *Galutinio modelio klasifikacijos ataskaita*

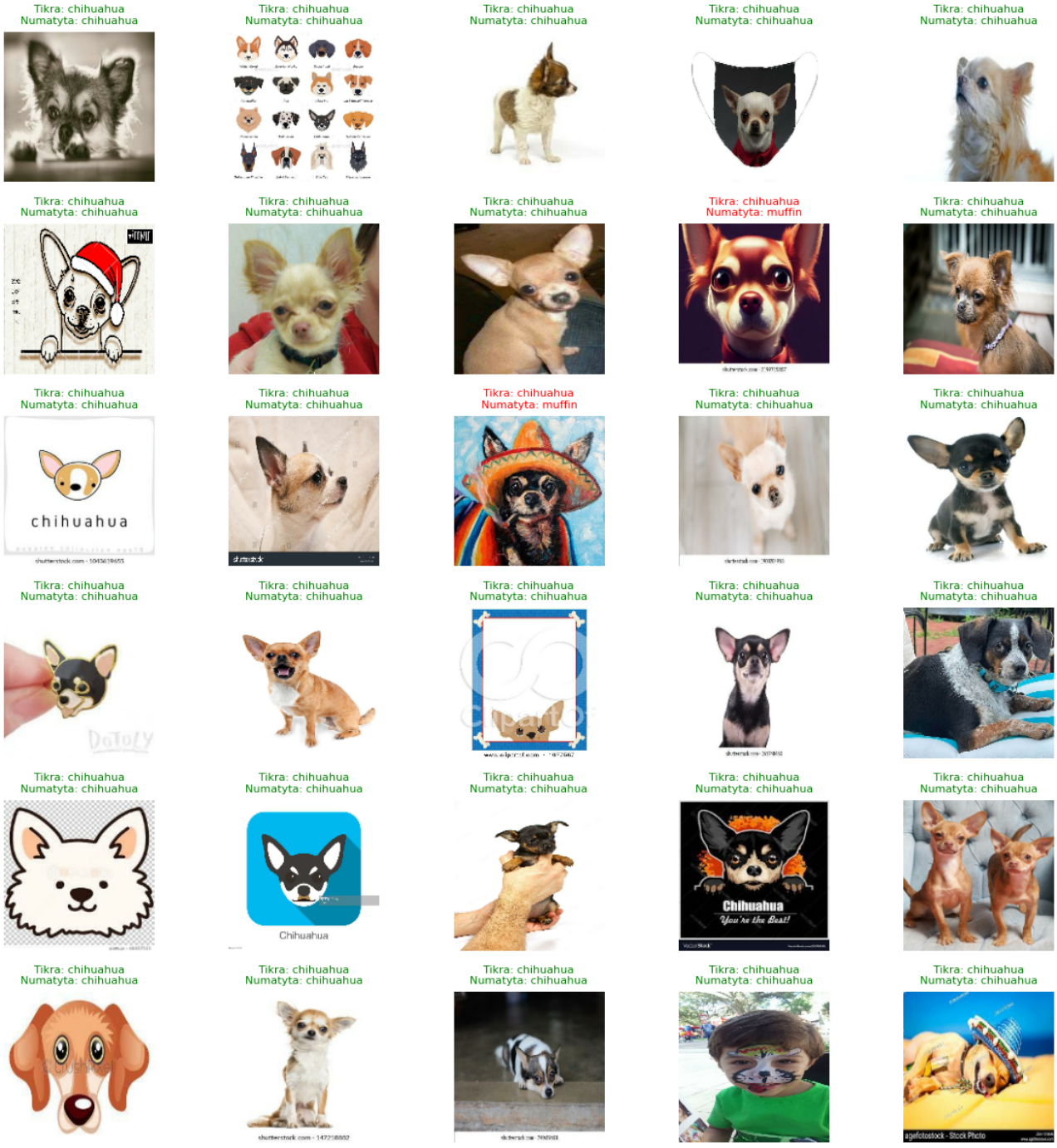
Klasė	**Precision**	**Recall**	**F1-score**	**Support**
Chihuahua	0,92	0,94	0,93	337
Muffin	0,92	0,90	0,91	256
Bendras tikslumas (Accuracy)		92,24%		

Kaip matome (žr. 6 lentelėje), modelio tikslumas siekia net 92,24%. Abiejų klasių rodikliai išlieka panašūs, o `chihuahua` klasė pasižymi šiek tiek didesniu `recall`, kas reiškia, jog modelis dažniau teisingai atpažįsta šios klasės objektus. Tokie rezultatai rodo, kad modelis efektyviai skiria abi klases.



4 pav. *Galutinio modelio klasifikavimo matrica*

Kaip matome (žr. 4 pav.), modelis abu objektų tipus klasifikuoja gana tiksliai. Iš 337 `chihuahua` vaizdų teisingai atpažinti 317, o iš 256 `muffin` teisingai atpažino 230. Klaidingai priskirtų atvejų skaičius yra nedidelis, kas rodo stabilų modelio veikimą ir gerą klasių atskyrimą.



5 pav. *Atsitiktiniai testavimo aibės pavyzdžiai su tikromis ir numatytomis klasėmis*

Kaip matome (žr. 5 pav.), dauguma pavyzdžių buvo klasifikuoti teisingai. Jie pažymėti žalia spalva. Klaidingai atpažinti atvejai pažymėti raudonai, tačiau jų skaičius nedidelis.

Išvados

1. Darbo metu buvo sukurti ir ištirti keli konvoliuciniai neuroniniai tinklai, skirti dvejetainiam vaizdų klasifikavimo uždaviniui, atskirti keksiukus nuo šuniukų.
2. Duomenų rinkinys iš Kaggle buvo suvienodintas ir padalintas į mokymo, validacijos ir testavimo aibes santykiu 80:10:10, iš viso panaudota 5917 vaizdų.
3. CNN architektūra buvo sudaryta iš konvoliucinių, „pooling“, „batch normalization“ ir „dropout“ sluoksnių, o tinklo sudėtingumas kito nuo 2 iki 4 konvoliucinių blokų.
4. Eksperimentai parodė, kad didesnis sluoksnių skaičius ir mažesnė „dropout“ reikšmė pagerina modelio tikslumą bei sumažina paklaidą.
5. „Batch Normalization“ kai kuriais atvejais pablogino rezultatus, greičiausiai dėl per didelio svorių suvienodinimo mažesniame duomenų rinkinyje.
6. Geriausias modelis (4 konvoliuciniai blokai, be „Dropout“ ir „BatchNorm“) pasiekė 91,54% validacijos tikslumą ir 92,24% testavimo tikslumą.
7. Klasifikavimo matrica parodė, kad abi klasės atpažįstamos panašiai tiksliai, o klaidų skaičius yra nedidelis.
8. Atsitiktinių testavimo pavyzdžių vizualizacija patvirtino, kad modelis teisingai klasifikuoja daugumą vaizdų, o klaidingų atvejų skaičius yra mažas.
9. Tyrimas parodė, jog net santykinai paprasta CNN architektūra gali pasiekti aukštą klasifikavimo tikslumą, jei tinkamai parinkti hiperparametrai ir atlikta normalizacija.
10. Gauti rezultatai rodo, kad konvoliuciniai neuroniniai tinklai yra itin efektyvūs net ir mažesnėse duomenų aibėse, kai taikomos tinkamos reguliavimo priemonės.

Literatūros sąrašas

[1] Samuel Cortinhas, *Muffin vs Chihuahua Image Classification Dataset*, Kaggle, 2023. Prieiga internetu: `https://www.kaggle.com/datasets/samuelcortinhas/muffin-vs-chihuahua-image-classification/data`

[2] [Prof., Dr. Olga Kurasov], *Dirbtinio intelekto pagrindai: paskaitų skaidrės*, Vilniaus universitetas, Matematikos ir informatikos fakultetas, 2025. Vidinė mokomoji medžiaga.

[3] OpenAI. *ChatGPT (GPT-5)* — pagalba klaidų taisymui ir terminų vertimui.